EVOLUTION OF UNSUPERVISED LEARNING

MODELLING DISTRIBUTIONS

Given a bunch of samples x_i, we try to model it into **parametric family of probability distribution** P_θ which can be found using maximum likelihood estimate

$$\max_{\theta \in R^d} \frac{1}{m} \sum_{i=1}^{m} \log P_\theta(x^{(i)})$$

If we assume that this x is large and models an imaginary probability distribution P_r , we try to make out distribution be as close to this. This is usually done through minimizing Kullback-leibler divergence $KL(P_r || P_\theta)$

However, the problem lies when the real distribution lies in a **low dimensional manifold** and the KL divergence comes out to be infinity as we are unable to model it as P_{theta} becomes 0

This is usually resolved by adding noise to it which makes it blurry however

SAMPLING METHODS WITH VARIATIONAL AUTO-ENCODERS AND GENERAL ADVERSARIAL NETWORK

We instead try to sample **z ~ P(z)** and **x ~ g_{theta}(z)**

Variational autoencoders:

They learn the distribution as a comination of normal distributions. Therefore they learn the mean and variance parameters to sample from it. However, they still involve noise

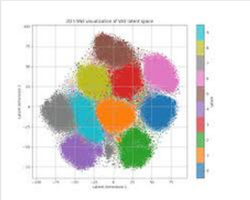
General adversarial networks:

They learn a generator which generates a real-looking sample from a latent space and a discriminator which tries to distinguish a real data point from a fake data point.

Training GANs is a very unstable process however because of **"mode collapse"** which will be elaborated later

OBJECTIVE

The objective then is to use divergence functions such that $\theta \rightarrow P_\theta$ is continuous



latent space of a VAE

ANALYSIS OF DIFFERENT DIVERGENCE FUNCTIONS

TOTAL VARIATION (TV)

$$\delta(P_r, P_g) = \sup_{A \in \Sigma} |P_r(A) - P_g(A)|$$

JENSON-SHANNON (JS)

$$\mathrm{JS}(P_r, P_g) = \mathrm{KL}(P_r \,\|\, P_m) + \mathrm{KL}(P_g \,\|\, P_m)$$

KULLBACK-LEIBLER (KL)

$$\mathrm{KL}(P_r \,\|\, P_g) = \int \log\left(\frac{P_r(x)}{P_g(x)}\right) P_r(x)\, d\mu(x)$$

EARTH-MOVER (WASSERSTEIN)

$$W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} \mathbb{E}_{(x,y)\sim\gamma}\left[\|x - y\|\right]$$

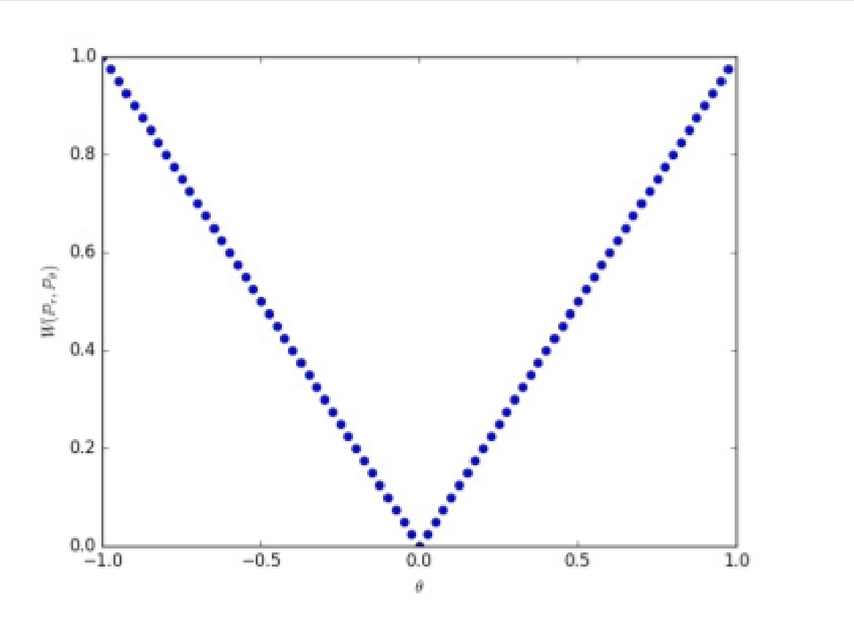
$$Z \sim \mathcal{U}[0,1], \quad P_0 = (0, Z), \quad P_\theta = (\theta, Z)$$

$$\mathrm{KL}(P_0 \,\|\, P_\theta) = \mathrm{KL}(P_\theta \,\|\, P_0) = \begin{cases} \infty, & \theta \neq 0 \\ 0, & \theta = 0 \end{cases} \qquad \delta(P_0, P_\theta) = \begin{cases} 1, & \theta \neq 0 \\ 0, & \theta = 0 \end{cases}$$

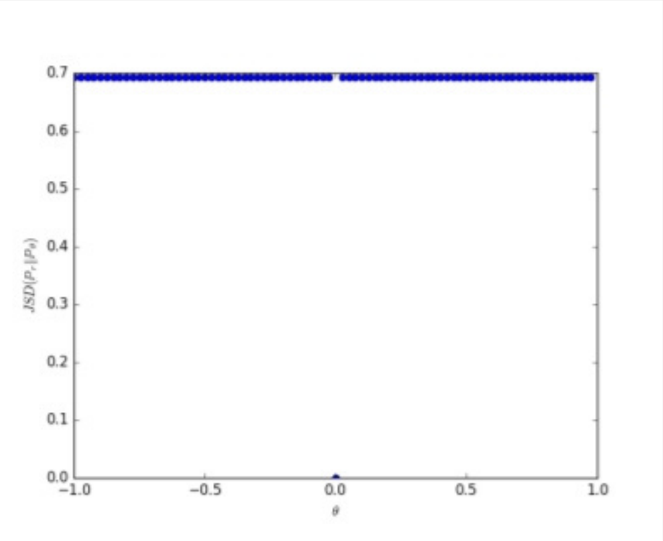
WASSERSTEIN VS JS

$Z \sim \mathcal{U}[0,1], \quad P_0 = (0, Z), \quad P_\theta = (\theta, Z)$

$$Warsesstein : W(P_0, P_\theta) = |\theta|$$



$$\mathrm{JS}(P_0, P_\theta) = \begin{cases} \log 2, & \theta \neq 0 \\ 0, & \theta = 0 \end{cases}$$



WASSERSTEIN AS A CRITIC

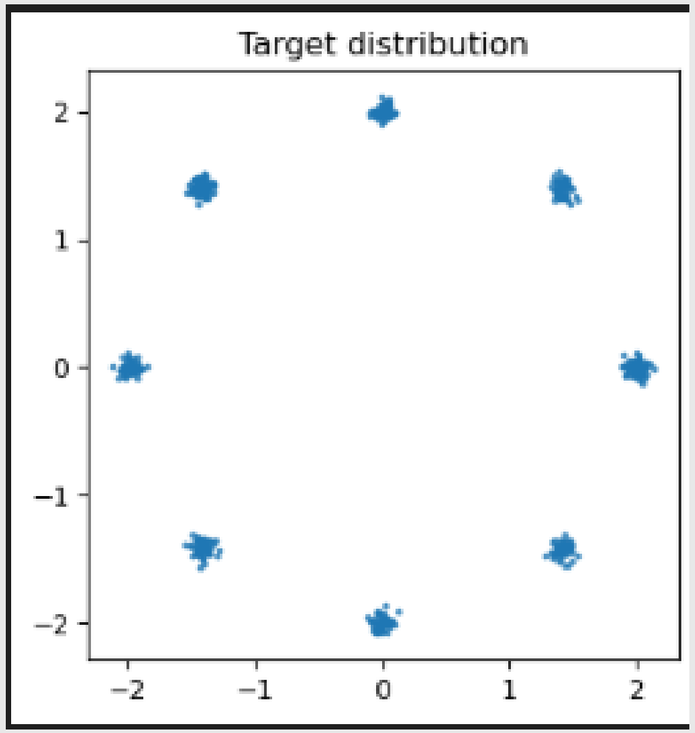
WASSERSTEIN CRITIC

- Measures distance as the minimum cost of transporting mass between real and generated distributions
- Uses a critic instead of a discriminator that outputs real-valued scores
- Enforces a 1-Lipschitz constraint on the critic (via weight clipping)
- Produces smooth, continuous gradients even when distributions do not overlap
- Leads to more stable training with fewer oscillations
- Loss correlates well with sample quality
- Reduces mode collapse

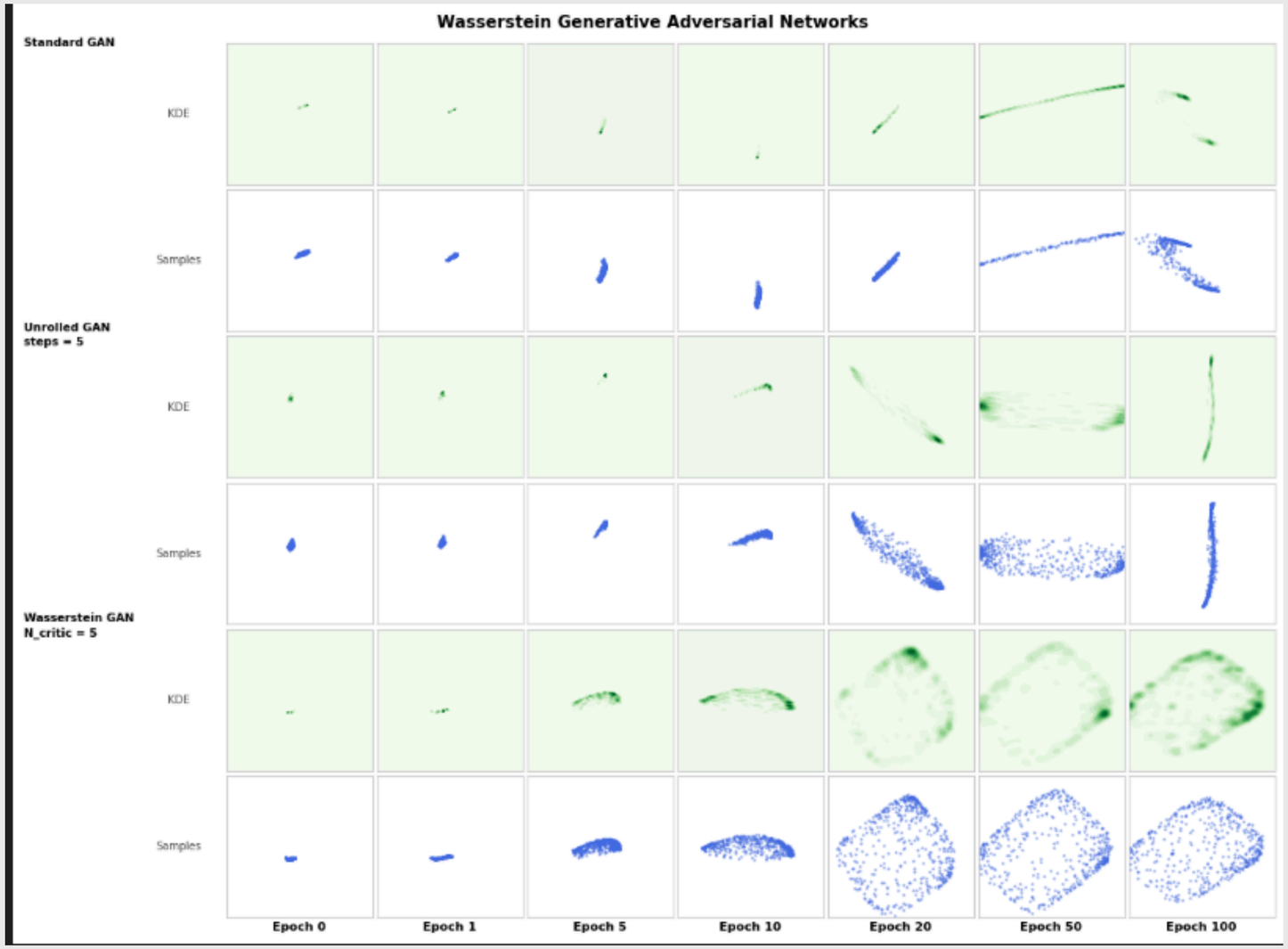
JS DISCRIMINATOR

- Measures similarity using probability overlap between real and generated distributions
- Uses a discriminator that outputs probabilities (via sigmoid)
- No Lipschitz constraint; standard classification objective
- Gradients vanish when distributions do not overlap
- Leads to unstable training and oscillations
- Loss does not correlate with sample quality
- Prone to mode collapse

DIFFERENT GANS LEARNING 8 NORMALS AROUND A CIRCLE



This is a distribution of **8 normals around a circle** which the GANs try to learn
We can see WGAN eventually learns it the best



model params:

2 layers

8 hidden units

dist params:

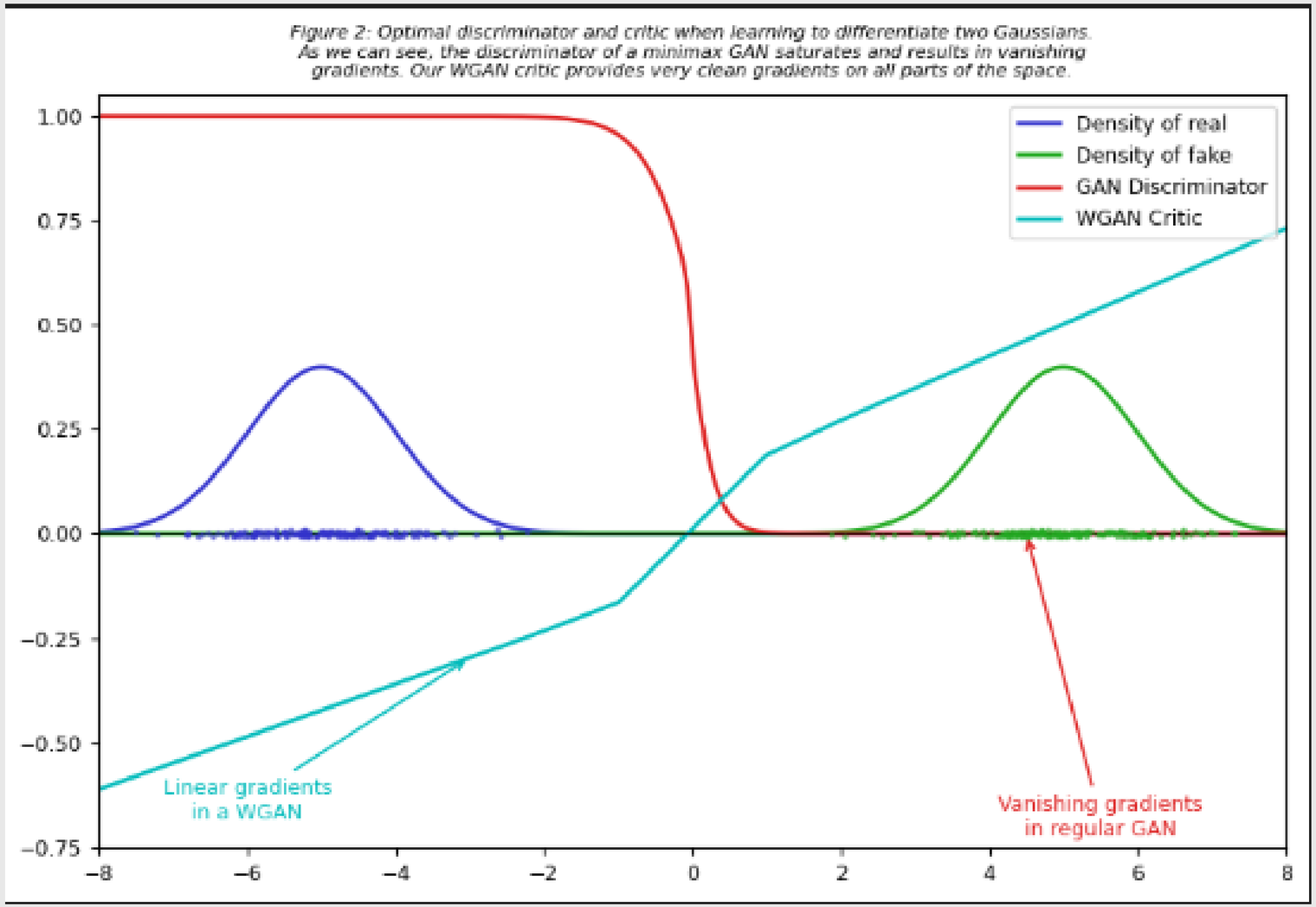
r:2.0

std:0.05

GRADIENT ANALYSIS OF GANS LEARNING 2 NORMALS IN X AXIS

Distribution details:
real distribution modelled as a normal with mean = -5, std=1
fake distribution modelled as a normal with mean = 5, std=1

Gradient analysis
GANs use sigmoid hence they have low gradient away from 0 causing mode collapse or unstable training
WGANs use clipped Wasserstein hence they have a very **stable** gradient across the entire x axis pushing for better training updates



DATA PREPARATION

PROBLEMS

Extremely huge dataset: because the dataset is so huge (3 mil) and we were unable to download it from the stanford website(which is slow) and also because working on it in kaggle will not be possible

Large images compressed in preprocessing: because the dataset is so large authors converted it to 64*64 pixel images to process it. This was done everytime

Loading the dataset into kaggle: because we work with kaggle notebooks, we need a way to load data (and optionally skip preprocessing too if possible)

SOLUTION FOR IT

We use a 10% sample which was initially taken from https://www.kaggle.com/datasets/jhoward/lsun_bedroom *(It says 20% in title but is 10% as mentioned in description)* which is 300k images

We exploit preprocessing by doing it once with a script which preprocessed the data and converts it to a much smaller file of just 1.2GB which is very easy to work with

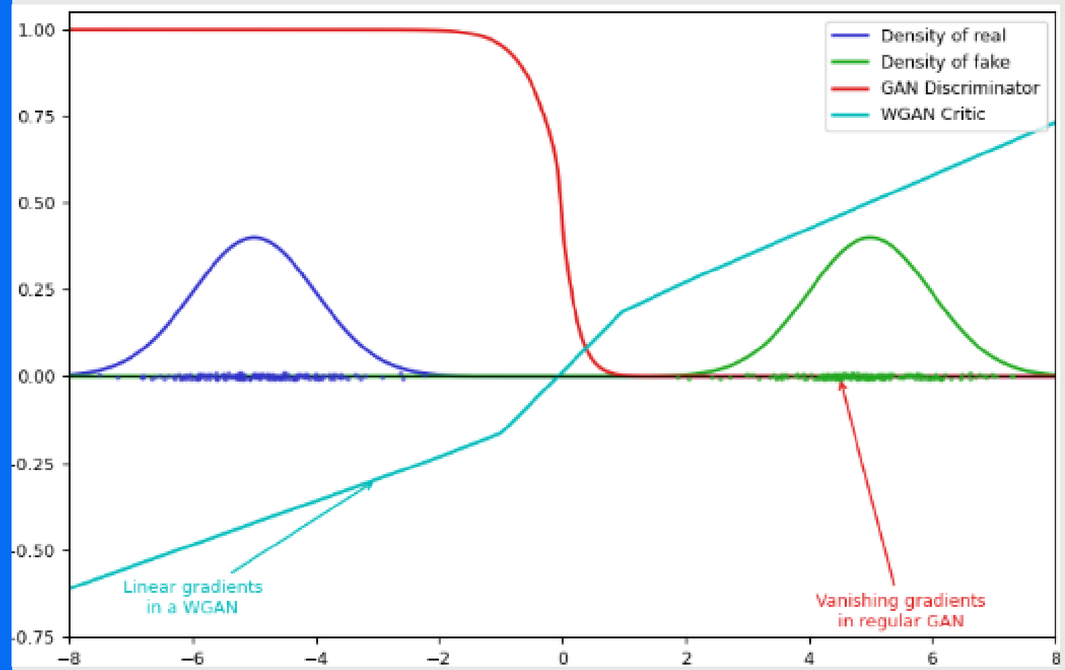
Final approach was that we preprocessed the data locally 8.9 → 1.2 GB and make a public kaggle dataset which we simply load and reuse it saving an insane amount of time in download

Final dataset: https://www.kaggle.com/datasets/udaykalyansreenivasa/lsun-bedroom-64x64-10perc

LIMITATIONS OF GAN

1. Training instability:

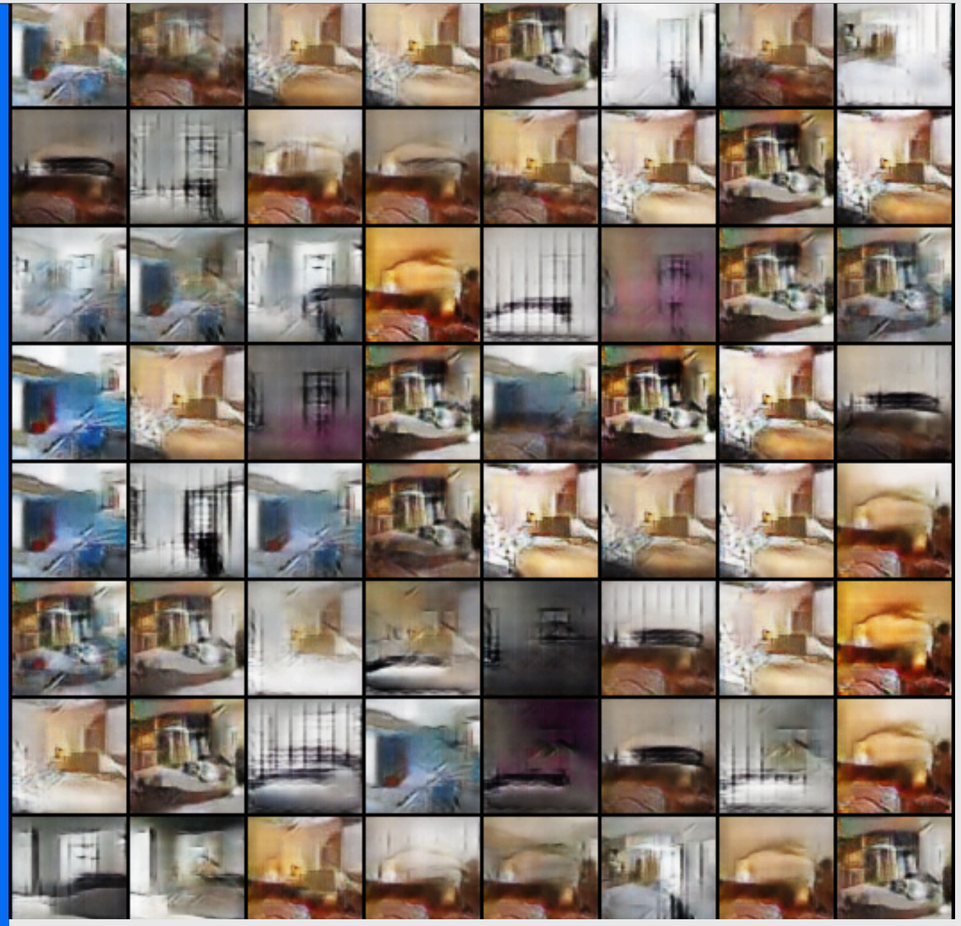
Standard GAN training is notoriously unstable. The generator and discriminator need to improve at roughly the same pace — if the discriminator gets too good too fast, it outputs near-zero gradients and the generator stops learning entirely. As you can see in the both extremes in the figure.



LIMITATIONS OF GAN

2. Mode Collapse:

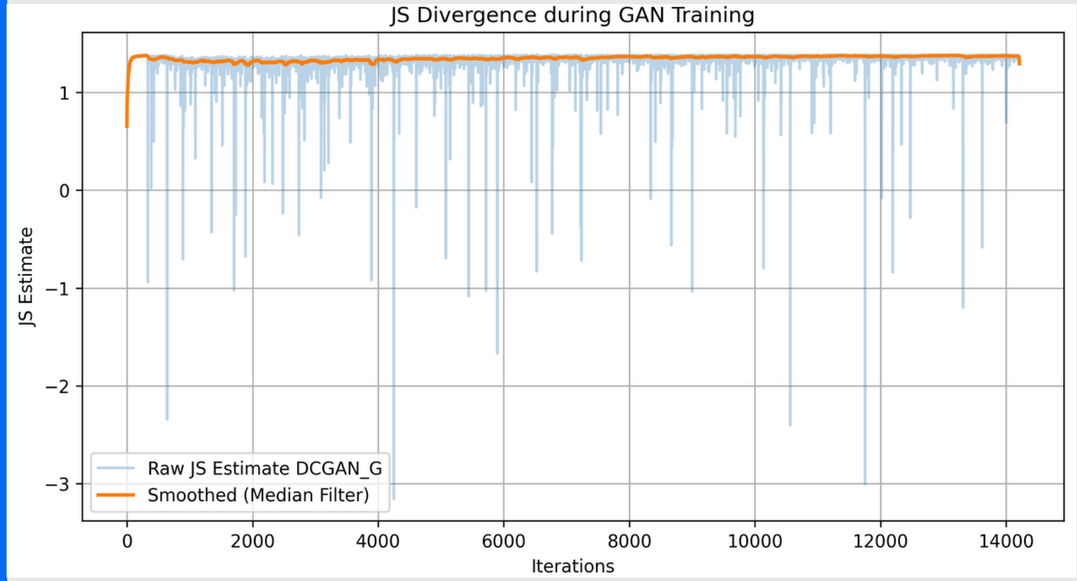
The generator finds a small set of outputs that reliably fool the discriminator and just keeps producing those, ignoring the full diversity of the real data. For example, if training on bedrooms, it might only ever generate one type of room layout.



LIMITATIONS OF GAN

3. No Meaningful Loss Metric:

The BCE loss value during training tells you almost nothing about image quality. A low discriminator loss doesn't mean good images — you have to visually inspect outputs constantly to judge progress.



LIMITATIONS OF GAN

4. Nash Equilibrium Problem

GAN training is a two-player minimax game — both networks are simultaneously trying to beat each other. Reaching a true equilibrium is theoretically hard and often never actually happens in practice.

ADVANTAGES OF WGAN

1. Because of Wasserstein Distance the vanishing gradient problem gets solved, becasue it provides a smooth, continues curve which is meaning full everywhere, even if 2 distributions don’t completely allign.

2. JS loss focuses on getting the image exactly right beacuse of which it is forced into mode collapse (as we saw in figure 1). Wasserstein Distance first focuses on minimizing the distance and then getting specific beacuse of which the chances of mode collapse decrease significantly.

ADVANTAGES OF WGAN

3. The Wasserstein loss correlates with visual quality. As the loss goes down, images genuinely get better. This means you can monitor training progress numerically without constant visual inspection — a huge practical improvement.

4. The critic (discriminator) in WGAN is trained to completion before each generator update. This gives the generator a stable, accurate signal to learn from rather than chasing a moving target.

JS DIVERGENCE BEHAVIOR IN GANS

Understanding JS Divergence in GAN Training

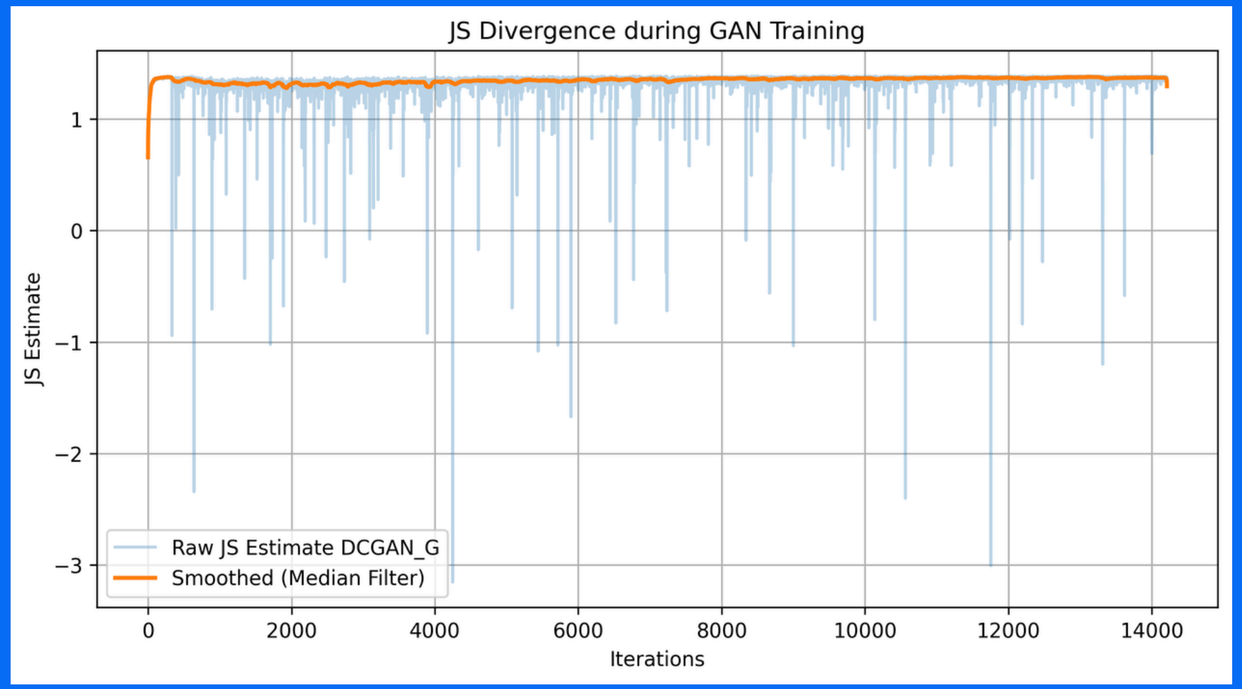
- Standard GANs optimize Jensen-Shannon (JS) divergence between real and generated distributions. The discriminator estimates this divergence during training
- Ideally: Better samples ⇒ Lower divergence ⇒ Decreasing loss
- But reality behaves differently. When distributions don’t overlap: JS divergence saturates (~1.38) and Gradients become uninformative
- This leads to weak learning signal for generator and unstable training dynamics
- Core idea: GAN loss may not reflect actual learning progress

$$L(D, g_\theta) = \mathbb{E}_{x \sim \mathbb{P}_r}[\log D(x)] + \mathbb{E}_{x \sim \mathbb{P}_\theta}[\log(1 - D(x))]$$

APPROACH

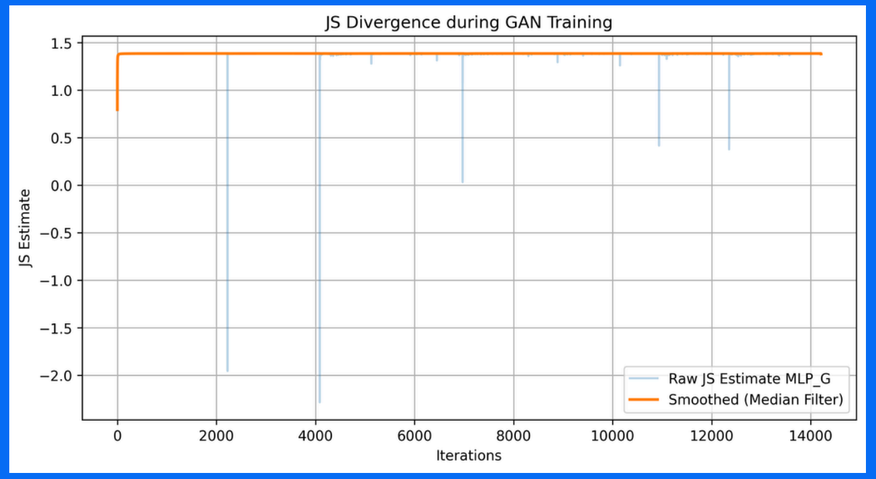
- Trained for 3 epochs only, 14K steps
- Trained multiple architecture combinations:
  - MLP Generator + MLP Discriminator
  - MLP Generator + DCGAN Discriminator
  - DCGAN Generator + DCGAN Discriminator
  - Computed JS divergence estimate using discriminator loss formulation
- Applied:
  - Median filtering to smooth noisy curves
  - Consistent training setup across experiments
- Objective:
  - Check whether JS divergence correlates with sample quality

RESULTS



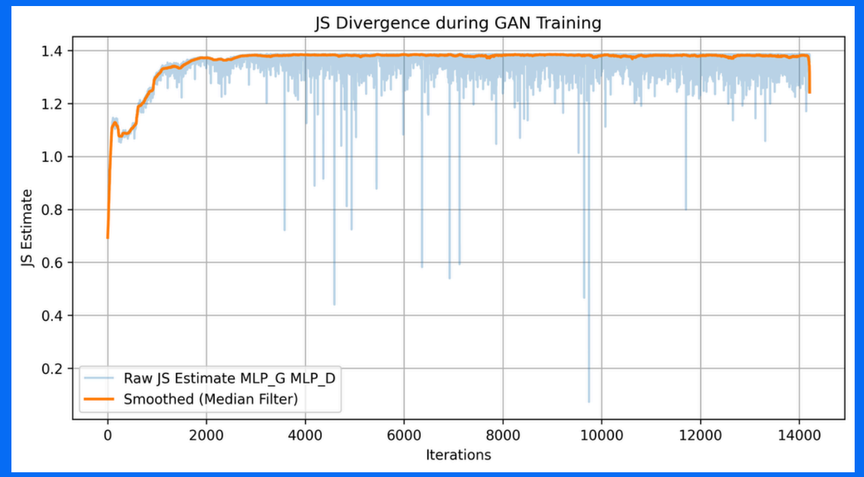
Standard GAN model with DCGAN generator and DCGAN discriminator

RESULTS



Standard GAN model with MLP generator and DCGAN discriminator

RESULTS



Standard GAN model with MLP generator and MLP discriminator

OBSERVATIONS

Plot 1: DCGAN Generator and DCGAN Discriminator

- In all three setups, the smoothed JS estimate rapidly rises and plateaus (~1.3–1.4) early in training. After this point, there is almost no meaningful decrease.
- Interpretation: The model reaches a region where JS divergence stops providing useful gradient

Plot 2: MLP Generator and DCGAN Discriminator

- The blue curves show large, frequent negative spikes. These spikes are high variance artifacts of minibatch estimation. They do not reflect actual training progress
- Key insight: Raw JS estimates are unstable and unreliable without smoothing

Plot 3: MLP Generator and MLP Discriminator

- Despite training continuing for ~14k iterations, JS divergence remains flat (after smoothing) and no consistent upward or downward trend
- Interpretation: The loss does not indicate whether the generator is improving

WGAN Core Idea

Problem with standard GAN :

- Standard GAN minimizes Jensen–Shannon divergence
- When Pr and Pg lie on low-dimensional manifolds, they often do not overlap
- Then JS divergence becomes constant
- Generator receives almost zero gradient

WGAN idea :

- Replace JS divergence with Wasserstein distance (Earth Mover Distance)

Definition :

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y)\sim\gamma}\left[\, \|x - y\| \,\right]$$

Measures how much distribution mass must move and how far

- Minimum cost required to transport mass from generated distribution Pg to real distribution Pr

WGAN Training Algorithm

Key changes from GAN:

- Discriminator → Critic
- Remove final sigmoid
- Critic approximates Wasserstein distance between Pr and Pg
- Train critic more often so it better estimates Wasserstein distance ,Use RMSProp
- Clip critic weights to enforce 1-Lipschitz constraint
- Weight clipping enforces the 1-Lipschitz constraint required by Kantorovich–Rubinstein duality

$$\max_D \mathbb{E}_{x\sim P_r}[D(x)] - \mathbb{E}_{x\sim P_g}[D(x)]$$

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α, the learning rate. c, the clipping parameter. m, the batch size. n_{critic}, the number of iterations of the critic per generator iteration.

Require: : w_0, initial critic parameters. θ_0, initial generator's parameters.

while θ has not converged **do**
  for $t = 0, \ldots, n_{\text{critic}}$ **do**
    Sample $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$ a batch from the real data.
    Sample $\{z^{(i)}\}_{i=1}^m \sim p(z)$ a batch of priors.
    $g_w \leftarrow \nabla_w [\frac{1}{m}\sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m}\sum_{i=1}^m f_w(g_\theta(z^{(i)}))]$
    $w \leftarrow w + \alpha \cdot \text{RMSProp}(w, g_w)$
    $w \leftarrow \text{clip}(w, -c, c)$
  end for
  Sample $\{z^{(i)}\}_{i=1}^m \sim p(z)$ a batch of prior samples.
  $g_\theta \leftarrow -\nabla_\theta \frac{1}{m}\sum_{i=1}^m f_w(g_\theta(z^{(i)}))$
  $\theta \leftarrow \theta - \alpha \cdot \text{RMSProp}(\theta, g_\theta)$
end while

Why WGAN is More Stable

Standard GAN

- JS divergence saturates
- discriminator becomes too strong
- gradients vanish
- training oscillates / collapses

WGAN

- Wasserstein distance changes smoothly
- critic improves gradient quality
- gradients remain useful
- training stays stable

Why mathematically?

- Continuous almost everywhere
- differentiable almost everywhere
- meaningful even when supports do not overlap